

Milestone 1
Personal Smart Locker

a) While control of servo motors is easy if you use external Arduino libraries, it is important to understand how they work and how to implement them yourselves. For this project, you'll need to implement your own control of a servo motor without using external Arduino libraries. Use resources online to learn about how servos are controlled and write a short explanation along with a short pseudocode demonstrating how you'll turn on your servo, control it to a specific angle, and turn it off as needed. Note that you'll need to monitor and control the rest of your system at the same time, so this code should not be blocking.

- **How we will control the servo**

- Hobby servo motors typically adjust their angle based on PWM input from the signal wire. Reading through [1] I found that hobby servo motors have three wires running into them, the ground, the 5v voltage as mentioned in the spec and the signal wire. We have to generate a 5v PWM signal that is calibrated with the potentiometer that lies in the servo motor as well as send a constant 5v input into the servo motor to control it without using the built in arduino library.
- We looked through frequencies that were recommended and the recommended range is 40-400hz. We decided that we will be using a 50 Hz frequency, and our max possible period for a single pulse will be 20 ms long, as that is the cycle length. Our pulse signals will most likely be based on typical servos whose pulse width lies between 1-2ms but that inevitably depends on our calibration testing. The reason we decided on 50hz is simply because it is a trade off between precision and latency. If we choose a higher frequency such as 400hz or higher we would have near instant ($\sim <2\text{ms}$ [1]) response times. Vice versa if we choose 50hz we would have a lot more precision in terms of the angle of the servo. In our past experience working with servo motors we have experienced a lot of issues with accuracy in terms of hobby motors and the need to constantly recalibrate them if you are working with higher frequencies. We recognize the need to calibrate them initially, but playing with the risk of the angle of our lock not working properly and our safe not opening or closing when requested would be a huge detriment to our goals. We also recognized that we are not in the field of Avionics or any field that needs instant response from their servos or motors, and that the trade off for precision is favorable for our scope.
- We are planning to employ servos for the locking mechanism. We will have two positions our servo motor will be in: locked or unlocked. Lots of thought goes into where these two positions are: we do not want to overly stress our servo from moving from positions very far away from each other (such as 0 and 180 degrees) because that can result in inaccuracies from the internal potentiometer

and gears of the servo over time as they are traveling a longer duration increasing the tendency for inaccuracies. Furthermore we do not want the endpoints too close to each other where the hobby servo doesn't have the space to move and will result in the internal potentiometer and gears developing inaccuracies. Through our research and past experience we believe the sweet spot for our angles is 60 degrees for locked, and 150 degrees for unlocked, giving the servo enough room to engage and disengage. Our pulse width, however, is yet **to be determined**, as it differs for each motor, so we will be determining this in milestone 2 where we actually test the pulse widths and where they bring the motor to. Our plan for calibration is to find the PWM signal for 60 deg then 150 and then finally oscillate between the two for a long duration and see the inaccuracies it develops and then see what we can improve.

- Our design must be non-blocking and not interfere with the rest of our system. Our design is nonblocking because the CPU never sits and waits in a delay loop to generate the servo pulses. Instead, a hardware timer generates the 50 Hz timing in the background, and our main loop keeps running to scan the keypad and run the state machine at the same time. This way, we don't need to stop the system and have any other components waiting on delays. Also we looked into multithreading possibilities but that will not work on the ATmega board without implementing an RTOS but since I believe we are not allowed to use external libraries that would be trivial. Also we would probably have to create our own flash and ram for that capability. Additionally if you look at the image that I attached below, you can see the pin layout of the ATmega. The hardware timer can generate a PWM waveform on an OC1x pin, and in Fast PWM we can set the period using ICR1 while using OCR1A (or OCR1B) to set the pulse width.

ATMega328P and Arduino Uno Pin Mapping

Arduino function					Arduino function
reset	(PCINT14/RESET) PC6	1	26	PC5 (ADC5/SCL/PCINT13)	analog input 5
digital pin 0 (RX)	(PCINT16/RXD) PD0	2	27	PC4 (ADC4/SDA/PCINT12)	analog input 4
digital pin 1 (TX)	(PCINT17/TXD) PD1	3	28	PC3 (ADC3/PCINT11)	analog input 3
digital pin 2	(PCINT18/INT0) PD2	4	29	PC2 (ADC2/PCINT10)	analog input 2
digital pin 3 (PWM)	(PCINT19/OC2B/INT1) PD3	5	24	PC1 (ADC1/PCINT9)	analog input 1
digital pin 4	(PCINT20/XCK/T0) PD4	6	23	PC0 (ADC0/PCINT8)	analog input 0
VCC	VCC	7	22	GND	GND
GND	GND	8	21	AREF	analog reference
crystal	(PCINT6/XTAL1/TOSC1) PB6	9	20	AVCC	VCC
crystal	(PCINT7/XTAL2/TOSC2) PB7	10	19	PB5 (SCK/PCINT5)	digital pin 13
digital pin 5 (PWM)	(PCINT21/OC0B/T1) PD5	11	18	PB4 (MISO/PCINT4)	digital pin 12
digital pin 6 (PWM)	(PCINT22/OC0A/AIN0) PD6	12	17	PB3 (MOSI/OC2A/PCINT3)	digital pin 11 (PWM)
digital pin 7	(PCINT23/AIN1) PD7	13	16	PB2 (SS/OC1B/PCINT2)	digital pin 10 (PWM)
digital pin 8	(PCINT0/CLKO/CP1) PB0	14	15	PB1 (OC1A/PCINT1)	digital pin 9 (PWM)

Digital Pins 11, 12 & 13 are used by the ICSP header for MOSI, MISO, SCK connections (Atmega168 pins 17, 18 & 19). Avoid low-impedance loads on these pins when using the ICSP header.

● Pseudocode for servo control

constants

```
servoPeriodMicroseconds = 20000 // 50 Hz period
servoMinPulseMicroseconds = 1000 // starting guess, calibrate per servo
```

```
servoMaxPulseMicroseconds = 2000    // starting guess, calibrate per servo
```

```
lockAngleDegrees = calibratedValue    // example 60 after testing
```

```
unlockAngleDegrees = calibratedValue    // example 150 after testing
```

state variables

```
isServoEnabled = false
```

```
timerPrescaler = chosenValue
```

```
cpuClockHz = fCpu    // CPU clock frequency in Hz
```

```
timerTicksPerMicrosecond = cpuClockHz / timerPrescaler / 1000000
```

```
targetServoPulseMicroseconds = 1500
```

```
targetServoAngleDegrees = lockAngleDegrees
```

```
lastServoCommandTimeMilliseconds = 0
```

helper functions

```
function clamp(value, minValue, maxValue)
```

```
    if value < minValue return minValue
```

```
    if value > maxValue return maxValue
```

```
    return value
```

```
function angleDegreesToPulseMicroseconds(angleDegrees)
```

```
    angleDegrees = clamp(angleDegrees, 0, 180)
```

```
    return servoMinPulseMicroseconds +
```

```
        angleDegrees * (servoMaxPulseMicroseconds - servoMinPulseMicroseconds) / 180
```

timer setup and control

```
procedure enableServoPwmOutput()
```

```
    if isServoEnabled
```

```
        return
```

```
    configure oc1aPin as output
```

```
    configure timer1 as Fast PWM with topRegister = icr1
```

```
    configure nonInvertingOutput on oc1aPin
```

```
    icr1 = servoPeriodMicroseconds * timerTicksPerMicrosecond    // sets 50 Hz period
```

```
    ocr1a = targetServoPulseMicroseconds * timerTicksPerMicrosecond // initial pulse width
```

```
    start timer1 with timerPrescaler
```

```
    isServoEnabled = true
```

```
procedure setServoPulseWidthMicroseconds(pulseMicroseconds)
```

```
    targetServoPulseMicroseconds = pulseMicroseconds
```

```
    if not isServoEnabled
```

```
enableServoPwmOutput()
```

```
ocr1a = targetServoPulseMicroseconds * timerTicksPerMicrosecond  
return immediately
```

```
procedure setServoAngleDegrees(angleDegrees)  
  targetServoAngleDegrees = clamp(angleDegrees, 0, 180)  
  pulseMicroseconds = angleDegreesToPulseMicroseconds(targetServoAngleDegrees)  
  setServoPulseWidthMicroseconds(pulseMicroseconds)  
  lastServoCommandTimeMilliseconds = currentTimeMilliseconds()  
  return immediately
```

```
procedure disableServoPwmOutput()  
  if not isServoEnabled  
    return
```

```
disconnect oc1aPin from timer1 output compare  
stop timer1 clock  
drive oc1aPin low  
isServoEnabled = false
```

high level commands

```
procedure commandLock()  
  setServoAngleDegrees(lockAngleDegrees)
```

```
procedure commandUnlock()  
  setServoAngleDegrees(unlockAngleDegrees)
```

power policy

```
procedure servoPowerPolicy()  
  if stateMachineSaysLatchIsHolding and holdingTorqueNotNeeded  
    disableServoPwmOutput()  
  else if currentTimeMilliseconds() - lastServoCommandTimeMilliseconds >  
servoIdleTimeoutMilliseconds  
    disableServoPwmOutput()
```

main loop

```
forever  
  scanKeypadNonblockingStep()  
  runStateMachineStep()  
  
if stateMachineRequestsLock  
  commandLock()
```

```
if stateMachineRequestsUnlock
  commandUnlock()
```

```
servoPowerPolicy()
manageSleepAndWakeStep()
```

Links/Citations:

[1] <https://blog.wokwi.com/learn-servo-motor-using-wokwi-logic-analyzer/>

b) For this project, you'll use a standard 12-key number pad to get input from the user. It would be very inefficient and expensive for these keypads to have a set of wires for each button, so instead each column and row has its own connection, requiring only 7 total connections to your system to use. However, this number of connections may still be more than you have available. Look into how to connect and use one of these keypads for your project and discuss a detailed plan for determining which button a user is pressing at any given time while using 4 or fewer input/output pins.

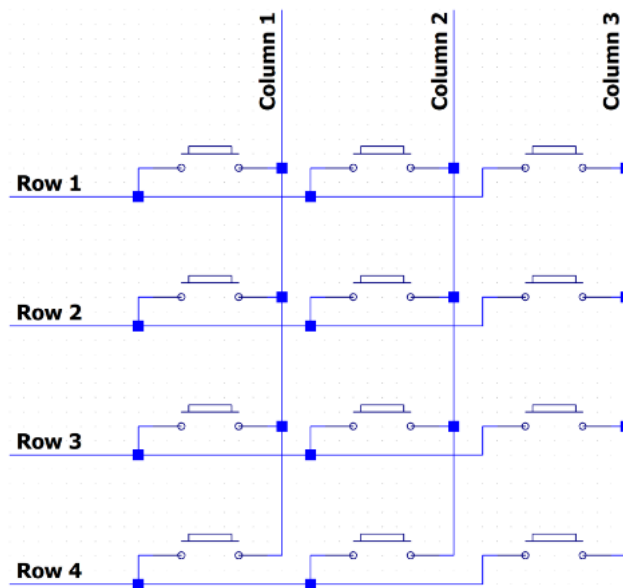


Figure 5: 12-Key Matrix Keypad Internal Connections

- **Determining which button is pressing at a time**
 - The 12 key pad is a 4x3 matrix (as shown in the figure). Each key has its own “coordinates” and no key shares the same row and same column. To detect the key, we can use matrix scanning, going through the columns, one at a time, reading the row lines. We’re configuring the row lines as inputs with pull ups, so

when nothing is being pressed, it's reading high. Then, we will pull one column low at a time whilst keeping the others high, and if a key in that column is pressed, the pressed switch connects that column to its row line, pulling the corresponding row input low. Using this, the index of the low row and low column identifies where the key is. The reason we "iterate" through the columns and not the rows is because there are less columns than rows.

- **Design approach**

- We can only use 4 or fewer input/output pins, so while the keypad will have 7 wires, we need to connect those 7 wires to external logic to end up communicating through 4 pins
 - We need to implement the keypad interface using shift registers so the microcontroller uses 4 pins total while still being able to read the 3 columns/ 4 rows.
 - We will need a clock shared by both registers, data out to the output shift register, a latch for the output shift register, and, obviously, data in from the input shift register.
 - The keypads 3 column wires will connect to 3 outputs of the serial out parallel out shift register, the keypads 4 row wires will connect to four inputs of the parallel in serial out shift register, and the row inputs will be held high by pull ups, defaulting to 1 when no key is pressed, as said earlier
 - We will output a pattern to the output shift register that drives column 1 low and the others high, then read the 4 inputs through the input shift register, and if any row bit reads low, we know a key is pressed in that row and that column. We then repeat this for the other columns.
 - We will use a fixed lookup table that maps to the keypad character 1..9, 0, *, and #. Row 0 is 1, 2, 3, row 1 is 4, 5, 6, row 2 is 7, 8, 9, and row 3 is *, 0, #, **as given to us in the project specification** (Figure 4: example of 12-Key Matrix Keypad buttons).
 - For debouncing, we will only accept a key if the same row/column is detected consistently through multiple scans. After accepting a key, we wait until all row inputs return high before accepting another press, so a press that's too long will only count as one input, not multiple
-

c) For your final design to be as cost and power efficient as possible, you will need to use the ATmega328P microcontroller chip rather than the Arduino Uno to save power and reduce cost of your design. While parts used to take this chip “offboard” are standard, you will have more options with how you enter and exit power saving mode to reduce the power used by the system when it is idle. Using the Atmega328P datasheet, explore different power saving modes and methods to enter and exit them and write up your intended mode and how you plan to implement it, justifying why you chose that mode and how it will work.

Table 9-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources							Software BOD Disable
	clk _{CPU}	clk _{FLASH}	clk _{IO}	clk _{ADC}	clk _{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other/O	
Idle			X	X	X	X	X ⁽²⁾	X	X	X	X	X	X	X	
ADC noise Reduction				X	X	X	X ⁽²⁾	X ⁽³⁾	X	X ⁽²⁾	X	X	X		
Power-down								X ⁽³⁾	X				X		X
Power-save					X		X ⁽²⁾	X ⁽³⁾	X	X			X		X
Standby ⁽¹⁾						X		X ⁽³⁾	X				X		X
Extended Standby					X ⁽²⁾	X	X ⁽²⁾	X ⁽³⁾	X	X			X		X

Notes: 1. Only recommended with external crystal or resonator selected as clock source.

2. If Timer/Counter2 is running in asynchronous mode.

3. For INT1 and INT0, only level interrupt.

• Design Options

- We know how to enter sleep as shown in the datasheet (Set the Sleep enable bit and execute the sleep instruction), but the main tradeoff to sleeping is that the deeper the sleep is, the less wake up mechanisms, even though we're saving more power.
- Table 9-1 from the datasheet (and pasted above) shows which wake-up sources are available in each mode. For our locker, the most important ones are the external interrupts and pin change interrupts because they can wake the chip when a user pressed a key on the keypad

• Design Choice

- Our choice of sleep mode is the power down mode. This mode is the mode that saves the most energy while still maintaining our necessary wake up sources. The datasheet table also shows that in this mode, the chip can still wake using INT0 or INT1, pin change interrupts, and the watchdog timer. We chose this sleep mode because it matches the behavior of a locker. When idle, we don't need to generate any servo pulses or scan the keypad. We only need to use the system when the user interacts or presses something. As soon as an interrupt

occurs during sleep, the datasheet states that the MCU wakes and runs the interrupt routine, then resumes from the instruction after Sleep. Because of this, we can safely store state machine variables in SRAM, sleep, then continue right where we left off when a user interacts and triggers it to wake.

- Our wake up method is that the pin change interrupt is the primary wake source, because they are asynchronous and described as usable for waking from deeper sleep modes. We can wire our user interaction signal so that a key press or wake button causes a logic transition which triggers the wake
- The datasheet also explains that a brown out detector can be disabled by software in some sleep modes to save power. This is just an idea, but we may disabled brown out detectors during sleep to reduce idle current even more.

● Implementation Plan

- First, we will decide when we are idle. If there has been no keypad activity for a set amount of time, and the servo isn't moving either, we will then enter idle mode and prepare for sleep mode.
- But, we need to prepare the chip to minimize power before the sleep, aka disabling systems we aren't using while idle using the Power Reduction Register. We will configure port pins to avoid waste. The datasheet warns that if an input buffer is enabled and the pin is floating , it can use excessive power, which we don't want, so we need to configure the pins
- We need to arm the wake source, so enable pin change interrupt masks for chosen wake pins (all buttons), so a user action triggers an interrupt.
- Then, we will enter power down sleep, setting SM2 SM1 SM0 to select power down mode, set SE, then execute sleep (instructions from datasheet). IF we choose to disabled brown detector, we will apply the CODS and CODSE timed sequence then execute sleep while CODS is active (again, instructions from the datasheet)
- Finally, we wake, then resume without blocking, so when the wake interrupt occurs, we will set wokeFromSleep = 1. The datasheet explains that after waking, the MCU executes the interrupt routine (I stated this earlier) and then resumes after the sleep instruction. This preserves SRAM. The main loop re-enables any systems we need, and we then continue how we left off, the keypad scan and state machine in play. This system stays nonblocking, because sleep isn't a busy wait, it's just the CPU stopping until it needs to be resumed.

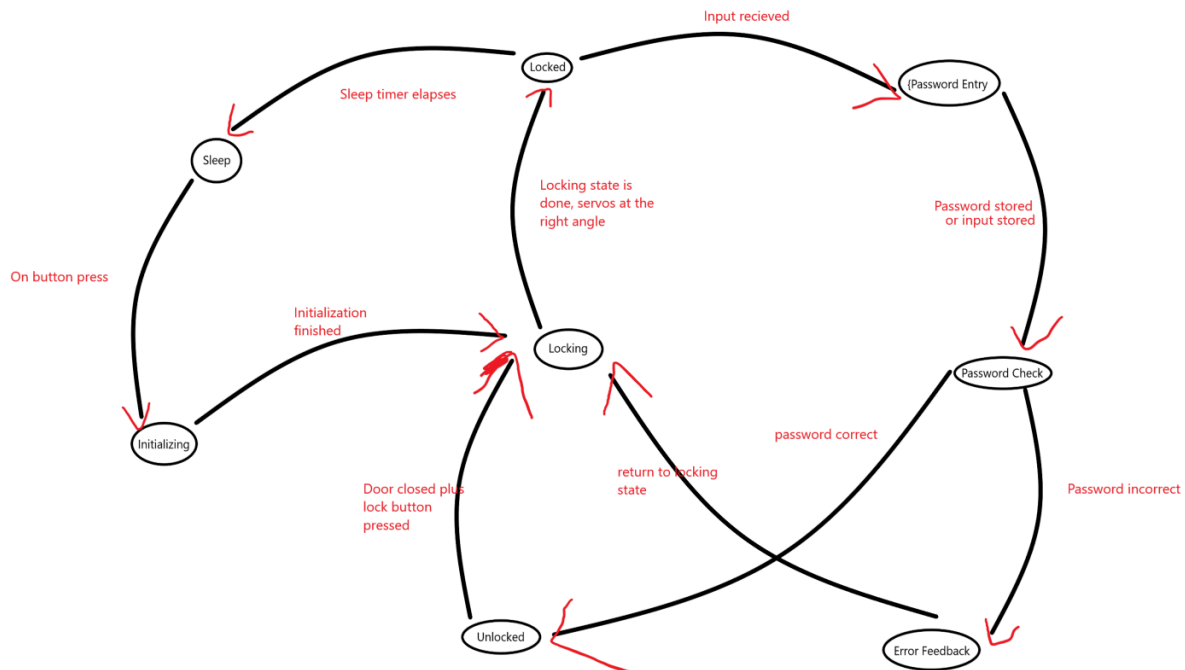
Links/Citations:

[2] <https://limewire.com/d/hGBDo#ZgHad400SS> (ATmega328P datasheet)

d) One of the more complex aspects of this project will be writing a program to control and monitor the entire system at once. For this scale of project, state machines can be used to ensure your states and transitions between states are completely defined and you can minimize the chance of jumping between states unexpectedly, as this may lead to bugs in your system. To get started on thinking about the logic behind your program, draw a detailed state transition diagram showing the different states of your system and how it will move in between states. This will ultimately become more complex as you add functionality to your system, but for now, consider a simplified system where you need to be able to:

- Power on the device when a button is pressed
- Enter a sleep mode when a certain amount of time has elapsed
- Receive input from a user for the password
- Check the password against the saved value
- Unlock the smart locker on a correct password
- Lock the smart locker when the lock button is pressed
- Display an error message/tone/etc. on an incorrect password

Additionally, write out a pseudocode that demonstrates how you could check your inputs and decide what state to transition to in this state machine.



Pseudocode:

enum lockerState

sleep
initializing
locked
passwordEntry
passwordCheck
unlocked
locking
errorFeedback

constants

inactivityTimeoutMs = chosenValue
errorFeedbackDurationMs = chosenValue
servoMoveTimeMs = chosenValue
maxPasswordLength = chosenValue

submitKey = '#'
clearKey = '*'

state variables

currentState = sleep
previousState = sleep

storedPassword = "1234" // placeholder for saved value
enteredPassword = "" // buffer while typing

inactivityDeadlineMs = 0
errorFeedbackEndTimeMs = 0
servoMoveEndTimeMs = 0

pendingLockRequest = false // set when lock button pressed
pendingUnlockRequest = false // set when password correct

helper functions

function nowMs() -> integer
function wakeButtonPressedEvent() -> bool
function lockButtonPressedEvent() -> bool
function getKeyEvent() -> char or none // returns one debounced key press per call
function isTimePast(deadlineMs) -> bool // true if nowMs() >= deadlineMs

procedure resetInactivityTimer()

inactivityDeadlineMs = nowMs() + inactivityTimeoutMs

```
procedure enterLowPowerSleep()  
  disableServoPwmOutput()  
  configureWakeSource()  
  goToSleepInstruction()
```

```
procedure commandServoLocked()  
  setServoAngleDegrees(lockAngleDegrees)
```

```
procedure commandServoUnlocked()  
  setServoAngleDegrees(unlockAngleDegrees)
```

```
main loop  
  forever  
    currentTime = nowMs()  
  
    wakeEvent = wakeButtonPressedEvent()  
    lockEvent = lockButtonPressedEvent()  
    key = getKeyEvent()  
  
    if wakeEvent or lockEvent or key != none  
      resetInactivityTimer()  
  
    stateChanged = (currentState != previousState)  
  
    if stateChanged  
      previousState = currentState  
  
    if currentState == initializing  
      enteredPassword = ""  
      pendingLockRequest = false  
      pendingUnlockRequest = false  
      resetInactivityTimer()  
  
    if currentState == locking  
      commandServoLocked()  
      servoMoveEndTimeMs = nowMs() + servoMoveTimeMs  
  
    if currentState == unlocked  
      commandServoUnlocked()  
  
    if currentState == errorFeedback  
      playErrorToneOrBlink()  
      errorFeedbackEndTimeMs = nowMs() + errorFeedbackDurationMs
```

```
if currentState == sleep
    enterLowPowerSleep()
```

```
switch currentState
```

```
case sleep
    if wakeEvent
        currentState = initializing
```

```
case initializing
    doOneTimeHardwareSetup()
    readStoredPasswordFromMemory()
    currentState = locking
```

```
case locking
    if isTimePast(servoMoveEndTimeMs)
        currentState = locked
```

```
case locked
    if isTimePast(inactivityDeadlineMs)
        currentState = sleep
    else if key != none
        currentState = passwordEntry
```

```
case passwordEntry
    if isTimePast(inactivityDeadlineMs)
        enteredPassword = ""
        currentState = sleep
```

```
else if key == none
    doNothing
```

```
else if key == clearKey
    enteredPassword = ""
    currentState = locked
```

```
else if key == submitKey
    currentState = passwordCheck
```

```
else if isDigit(key) and length(enteredPassword) < maxPasswordLength
    enteredPassword = enteredPassword + key
```

```
else
    doNothing
```

```
case passwordCheck
  if enteredPassword == storedPassword
    enteredPassword = ""
    currentState = unlocked
  else
    enteredPassword = ""
    currentState = errorFeedback

case errorFeedback
  if isTimePast(errorFeedbackEndTimeMs)
    stopErrorToneOrBlink()
    currentState = locked

case unlocked
  if isTimePast(inactivityDeadlineMs)
    currentState = sleep
  else if lockEvent
    currentState = locking
  else
    doNothing
```